

Spring Framework – Day 26

Annotations – Day 09: Java Configuration Approach

1. Declaring a Class as a Spring Bean

Let us take a simple class:

```
class Student {  
  
}
```

The question is: how do we declare this class as a Spring bean? Spring gives us several ways to do this. One of them is the XML approach.

XML Approach

application-context.xml

```
<beans>  
    <bean id="student" class="packagename.Student" />  
</beans>
```

This bean tag inside the XML file is called a **bean definition**.

2. Why Java Configuration?

We can also declare a bean using annotations, but annotations alone come with certain limitations. To address this, Spring provides another alternative: the **Java Configuration approach**.

3. Creating a Java Configuration Class

We create a normal class and mark it with the `@Configuration` annotation:

```
@Configuration  
class JavaConfig {  
  
}
```

- Just like the XML file, this class only holds bean definitions for configuration.
- That is why we mark the class with the `@Configuration` annotation.

In the XML approach, we write a bean definition so that the IoC container can create the object. In the Java Configuration approach, the programmer has to create the object explicitly, and hand it over to the IoC container, which then manages its dependencies and life cycle.

For this reason, we write a method with the following rules:

- The method modifier must be **public**.
- Since we are creating the object for the IoC container, the return type of the method should be the same as the class whose object we are creating.
- It is good practice to keep the method name the same as the class name.

```

@Configuration
class JavaConfig {

    @Bean
    public Student student() {
        Student ref = new Student();
        return ref;
    }
}

```

In XML, one bean definition is written using the `<bean>` tag. In the same way, in the `JavaConfig` class, a method acts as the bean definition. That is why we mark the method with the `@Bean` annotation.

In XML, we give a name to the bean using the `id` attribute. In the Java Configuration approach, this name is given through the bean method itself.

If you do not specify a name, the default name is taken as the method name. This is why it is better to keep the method name the same as the class name.

4. Calling the IoC Container

With XML configuration, we call the IoC container like this:

```

ApplicationContext context =
    new ClassPathXmlApplicationContext("xml path");

```

Since we are no longer using XML, let us look at the annotation-based way of calling the IoC container:

```

ApplicationContext context =
    new AnnotationConfigApplicationContext("package name");

```

Instead of passing the package name, we pass the Java Configuration class itself:

```

ApplicationContext context =
    new AnnotationConfigApplicationContext(JavaConfig.class);

```

5. Practical Demonstration

Let us test the concept with a small example made up of three files.

Student.java

```
package com.day26;

import org.springframework.stereotype.Component;

@Component
public class Student {

    Student() {
        System.out.println("Constructor");
    }

    public void work() {
        System.out.println("Working");
    }
}
```

JavaConfig.java

```
package com.day26;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class JavaConfig {

    @Bean
    public Student student() {
        return new Student();
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Main started");

        ApplicationContext context =
            new AnnotationConfigApplicationContext(JavaConfig.class);

        Student bean1 = context.getBean("student", Student.class);
    }
}
```